

CS 452: TC2 Documentation

Evan He, e7he, 20946651

Ian Zhao, i6zhao, 20988818

Abstract	4
Commit and SHA	4
Building	4
Train Controlling System	5
Command	5
TrainTrackState	6
Train	7
NavigationSystem	7
KinematicsSystem	7
PathFindingSystem	7
Routing	8
Stopping At The Target	9
Calibration Data	9
Sensor Ownership And Tracking	9
Tasks	11
TrainTrackTask (Priority 2)	11
MarklinEventListenerTask (Priority 0)	11
MarklinDispatchTask (Priority 2)	12
UIViewTask (Priority 2)	12
UIControllerTask (Priority 0)	12
Overall Design	12
Kernel Architecture	13
Components	13
K4	13
MCP2515	13
GPIO	13
UART	13
IO Server (for UART)	13
CAN Server	14
K3	17
StaticPriorityQueue	17
ClockServer	17
ClockNotifier	17
EventBlockedQueue[]	17
K2	18
RingBuffer	18

StaticStack	18
TaskIdAllocator	18
MessageControlBlock	18
NameServer	18
K1	19
TaskDescriptor	19
TaskScheduler	19
StackContext	19
RoundRobinQueue	19
MultiLevelQueue<T>	19
SlabAllocator<T, SIZE>	19
Control Flow	20
K4	20
CAN Interface	20
K3	21
GICC and GICD configuration	21
AwaitEvent	21
preIrqEntry	21
irqEntry(StackContext* sp)	21
Idle Task	22
K2	22
Message Queue Design	22
Send	22
Receive	23
Reply	23
K1	24
Task Creation	24
Context Switching	24
Kernel State	25

Abstract

TC2 contains:

A sophisticated train control system with a reservation system that plans multiple train's paths while avoiding collision at real time. Multiple trains can run on the same train track simultaneously with random destinations assigned.

Commit and SHA

Repo: https://git.uwaterloo.ca/e7he/cs452_kernel

Commit: 12d92499209b75b350cb4f9b847de49694481701

Building

The kernel image at "kitty_kernel.img" is built by running "make". There are no special requirements to build the project in the linux.student.cs environment.

The raw calibration data (stopping distance and speed measurements) is in the data/ subdirectory.

Train Controlling System

Command

tr <TrainId> <SpeedLevel>
Set the train speed.

st <TrackId>
Switch to track A or track B. This command must be performed at the beginning of the program. The track is defaulted to track B as track A is very finicky.

goto <TrainId> <SpeedLevel> <LocationName> <DistanceOffset>
Move the train to the location with an offset in millimeters. The location name is case sensitive. If the destination is a branch with positive distance offset, then the direction of the train depends on the switch direction.

wa <TrainId> <SpeedLevel>
Set the train into "Wandering" mode that travels to random destinations repeatedly.

TrainTrackState

```
struct TrainTrackState {
private:
    uint32_t currentTrack;
    std::array<Train, MAX_TRAIN_ID> trains{};
    TrackSet trackA{};
    TrackSet trackB{};
    std::array<SwitchState, NUM_SWITCHES> switches{};

public:
    TrainTrackState();

    void reset();

    Train& getTrain(TrainId id);
    SwitchState getSwitchState(SwitchId id) const;
    void setSwitchState(SwitchId id, SwitchState switchState);
    void setCurrentTrack(TrackId id);
    TrackId getCurrentTrackId() const;

    TrackNode& getTrackNodeById(TrackNodeId id);
    TrackNode* getTrackNodeByName(const char* name);
};
```

A TrainTrackState is a central state manager that keeps track of the state of track nodes, switch directions, and train's kinematics and navigation states.

Train

```
struct Train {  
    NavigationSystem navigation{};  
    KinematicsSystem kinematics{};  
    SensorPredictionSystem prediction{};  
};
```

Each train contains a kinematic state representing the motion of the train, a navigation state represents the intent of the train, and a prediction system that predicts the next triggered sensor.

NavigationSystem

Manual: The train is controlled by the user.

FindingPath: The train is finding a path to the destination.

Routed: The train is following the path.

Reversing: The train is reversing its direction.

KinematicsSystem

Lost: The train's kinematics state is unknown.

Tracked: The train's kinematics state is known.

PathFindingSystem

PathFindingSystem is responsible for reserving the track nodes and changing the switches' directions for the train every tick.

Routing

When a routing request arrives, the train enters `FindingPath` state in `src/user_tasks/train_track_handler.h`. Once the train has a tracked position, the path planner in `src/marklin/marklin_pathfinding.h` runs a Dijkstra search from the train's current estimated location to the destination. The search:

- explores all legal outgoing edges from each node,
- treats nodes reserved as the destination of another train as impassable,
- adds a penalty for nodes with queued reservation waiters, so the planner biases away from congested regions,
- accepts either the requested destination orientation or its reverse orientation, whichever is reached first.

If a path is found, the planner reconstructs it from the parent map, stores the ordered path nodes, and reserves each node. The reservation system consists of a FIFO queue for each node. A node is considered enterable by a train if the train is the next in the queue for that node. The destination node is locked separately and considered impassable for other trains.

If no forward path is found, the train sets a `needToReverse` flag, stops, reverses direction once its estimated speed reaches zero, re-seeds its tracked position on the reverse node, and tries to plan again. If no path exists even after reversing, the request fails unless the train is in wandering mode, in which case a new random destination is chosen.

While the train is moving, the planner continuously updates the route state. It also proactively sets branch directions for the next few path nodes ahead of the train, which avoids flipping a switch underneath the train.

Stopping At The Target

Stopping is handled by the path planner's runtime state machine, which moves each routed train between `Moving`, `Yielding`, `Arriving`, and `Idling`.

The planner recomputes the `enterableDistance` based on how many track nodes ahead it can enter every tick to decide when the train should yield or arrive at the destination.

- In `Moving`, the train keeps its commanded cruise speed.
- If the reserved distance ahead becomes too short to stop safely before an unenterable track, the train enters `Yielding` and is commanded to speed 0.
- If the destination itself is within stopping range, the train enters `Arriving` and is also commanded to speed 0.
- Once the estimated speed reaches zero in `Arriving` state, the train releases its reservations and returns to `Idling`.

There are also small overshoot and undershoot corrections near dangerous topology, to avoid directly stopping the train over a switch.

Calibration Data

Calibration data is stored in:

- `data/speeds.txt`
- `data/stopping_distance.txt`
- `src/marklin/marklin_measured_data.h` (computed from other data)

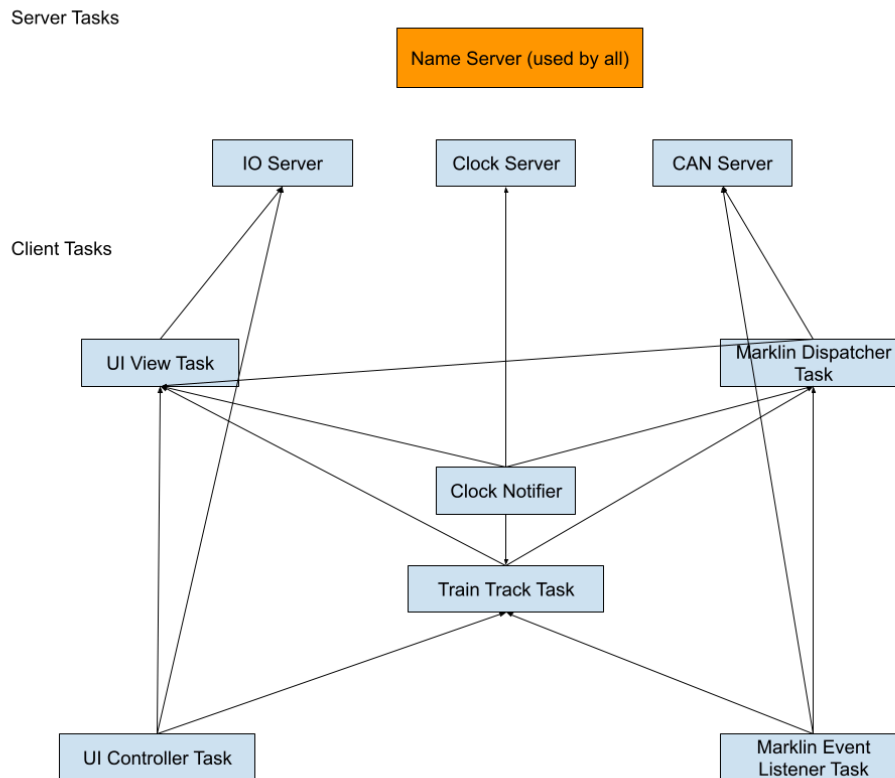
Sensor Ownership And Tracking

When a sensor event arrives, the train-control server must decide which train caused it. The ownership heuristic in `src/user_tasks/train_track_handler.h` uses three levels:

1. The train is currently holding the reservation on that sensor's node.
2. A train whose predicted next sensor matches the event.
3. A train currently in `Lost` kinematics state.

After ownership is resolved, the server logs the event, updates the train's kinematics from the measured inter-sensor distance and elapsed ticks, and finally predicts the next sensor and when it should occur.

Tasks



TrainTrackTask (Priority 2)

Train track server is the central train control system that manages the track and train states. It is responsible for all the train track state updates and path finding.

MarklinEventListenerTask (Priority 0)

Marklin event listener task listens to the CAN receive buffer, and decodes the CAN event data into user defined event type, and forwards the event data to the train track server and the marklin dispatch server..

MarklinDispatchTask (Priority 2)

Marklin dispatch server is responsible for managing all the network CAN messages. This includes waiting for the CAN response, and tracks the network latency.

UIViewTask (Priority 2)

The UI view server receives UI updating request messages from other tasks, and updates the screen pixel on demand. The UI prints out the system timer, switch states, sensor events, command history, and train states such as last triggered sensor name, estimated position, estimated speed, and estimated remaining distance to the destination.

UIControllerTask (Priority 0)

The UI controller task receives keyboard inputs and parses the inputs into commands. If the command is valid, it forwards to the train track server for execution.

Overall Design

The design is intentionally centralized. The train-control server is the single owner of train state and makes all routing and stopping decisions. This avoids excessive message patching to communicate shared state. The dispatcher isolates CAN protocol details. The event listener isolates raw Marklin feedback handling. The clock helper gives the control loop a regular tick. This keeps the logic easy to reason about:

- sensor events correct the model,
- timer ticks advance the model and reevaluate actions,
- the path planner decides whether the train should continue, yield, arrive, or reverse,
- the dispatcher converts those decisions into Marklin commands.

Kernel Architecture

Components

K4

MCP2515

MCP 2515 controller is a wrapper around the CAN bus protocol, responsible for translating the signals. CanIO interrupt ID 145 is enabled in the GIC.

GPIO

General Purpose Input Output. It provides hardware pins that trigger interrupts when certain events occur. For instance, MCP2515 interrupt sets PIN 17 to low and triggers the exception handler.

UART

Universal Asynchronous Receiver/Transmitter. It allows the user to communicate with the terminal. UartIO Interrupt ID 153 is enabled in the GIC.

IO Server (for UART)

IO Servers consist of an ioServerTask, a getcNotifierTask, and a putcNotifierTask following the server-notifier pattern.

ioServerTask contains a getcBuffer that stores all the inputs from the UART device to the client, a putcBuffer that stores all the output from the client to the UART device, and a getcWaitingQueue to store all the blocked waiting tasks that request to receive a char from the UART.

There are 4 IoServerMessageTypes: GetcRequest, PutcRequest, GetcNotify, PutcNotify.

GetcRequest:

The client requests to get the input from the UART device. If there's a char in the `getcBuffer`, then reply with the `getcBuffer`'s char, otherwise it blocks and adds to the `getcWaitingQueue`.

PutcRequest:

The client requests to send an input char to the UART device. If the device is available, then it sends to the UART device immediately. Otherwise, the `ioServerTask` stores the input at the `putcBuffer` and processes it later.

GetcNotify:

The `getcNotifierTask` awaits the `UART_RX` event to wait for the receive buffer available. If yes, it sends the `GetcNotify{input}` message to the `ioServerTask`. The `ioServerTask` checks if there's any task waiting in the `getcWaitingQueue`. If yes, then it replies to the task with the input value, otherwise, it adds to the `getcBuffer`.

PutcNotify:

The `putcNotifierTask` awaits the `UART_TX` event to wait for the transmit buffer available. If yes, it sends the `PutcNotify{}` message to the `ioServerTask`. The `ioServerTask` checks if there's any input waiting in the `putcBuffer`. If yes, it sends the input from the `putcBuffer` to the UART device, otherwise, it marks the UART device ready for input.

CAN Server

Main challenges in implementing the CAN servers.

Problem:

If task 1 calls a MCP operation that involves a SPI transaction, then no other concurrent task should call an MCP operation involving a SPI transaction, or else risk corrupting the SPI busline. This design constraint implies the kernel and the CAN notifier must not accidentally preempt the CAN servers's SPI transaction with another SPI transaction such as read MCP status or enable/disable the MCP interrupt. It is possible to bypass such limitations by carefully scheduling the tasks and managing the interrupt states, but such a solution leads to extreme complexity and it is difficult to reason about the correctness of the program.

Solution:

To avoid falling into the complexity trap, only the CAN server can call a MCP operation involving a SPI transaction. The program controls the CAN interrupt via GPIO interrupt registers instead of MCP interrupt registers, which bypasses the SPI transaction limitation, and hence can be used outside of the CAN server.

Question:

Why is there only a single CAN notifier in contrast to UART's input and output notifiers?

Answer:

There's no convenient way to determine the MCP receive and transmit buffer status without using SPI transactions. If we want to avoid falling into the complexity trap, then the MCP server is the only place where we can check the status.

In addition, both the read and transmit share the same SPI bus line to exchange bytes bidirectionally under the hood. Therefore, the CAN server needs only 1 CAN notifier as only either receive or transmit operations, but not both, can happen at a given time.

Implementation:

CAN servers consist of a `canServerTask` and a `notifierTask` with implementation similar to the IO Server. However, instead of checking the MCP IO status in the notifier task like the UART `ioNotifierTask`, the MCP IO status must be checked inside the `canServerTask`.

There are 3 `CanServerMessageTypes`: `ReceiveRequest`, `TransmitRequest`, `ReadyNotify`.

ReceiveRequest:

The client requests to get the input from the MCP2515. If there's a message in the `receiveBuffer`, then it replies with the `receiveBuffer`'s message; otherwise, it blocks and adds the client tid to the `receiveWaitingQueue`.

TransmitRequest:

The client requests to send a message to the MCP2515. If the transmit buffer is available, then it sends to the transmit buffer immediately. Otherwise, the canServerTask stores the input message at the transmitBuffer and processes it later.

ReadyNotify:

The notifier awaits the CanIO event from the GPIO PIN 17. If an event occurs, the irq_handler disables the GPIO event, and then the notifier sends a ReadyNotify{} message to the canServerTask. The canServerTask checks the MCP2515 interrupt flag to see if TX0, RX0, or RX1 is available. If any are true, it performs the corresponding action, similar to the IO server, and acknowledges the MCP2515 interrupt.

K3

StaticPriorityQueue

A min priority queue with fixed buffer size. The user can push a new item to the priority queue and pop the item with the lowest priority. Internally, the priority queue uses binary heap implementation with a $O(\log(n))$ complexity of push and pop, where n is the number of items in the priority queue.

ClockServer

A clock server is a task that handles time interface functions. Internally, it consists of a loop that `Receive()` a serialized Message object. A Message object consists of an enum type and a string buffer. The clock server checks the `Message.type`, which can be a tick update from the notifier, a Time request, or a Delay/DelayUntil request. In the last case, the server does not immediately reply to the task, instead maintaining delayed tasks in a priority queue sorted by the tick deadlines sent by the task. When the clock server receives a tick update, it keeps popping the priority queue until all the requests with expired deadlines have been replied to. The priority queue can contain a maximum of 128 tasks at a time.

ClockNotifier

A helper server that awaits the timer interrupt event, and sends the tick update message to its parent task, the clock server. This allows the clock server to “multiplex” timer interrupts and client requests by combining them both into a single `Receive()` call.

EventBlockedQueue[]

An array of cyclic queues indexed by the interrupt event id. All the tasks awaiting the same interrupt event type are enqueued into the same queue.

K2

RingBuffer

A FIFO cyclic queue with static size. The user can push a new item to the back, and pop an item from the front.

StaticStack

A LIFO stack data structure with static size.

TaskIdAllocator

The kernel task manager has gone through a complete overhaul compared to K1. Task id now consists of a generation number and an index number instead of a shared globalTidCounter. The index ranges from 0 - 127 that can be used to index into an array of TaskDescriptor in constant time. This eliminates the needs of slab allocators to manage the TaskDescriptors and task stack frames. To avoid task id duplication, each index has a generation counter that increments when the corresponding task id is freed.

MessageControlBlock

A message control block contains: the message to send, the size of the message to send, the receive buffer, the size of the receive buffer. Receive buffer can be used for both receiving the sending messages and receiving the replying message. Both the message to send and the receive buffer are non-owning pointers.

NameServer

A name server is a task that maps a string name to a task id and provides utilities for name lookup. Internally, it consists of a loop that Receive() a serialized Message object. A Message object consists of an enum type and a string buffer. The name server checks the Message.type, invokes the corresponding handler, then finally Reply() with the query result. We chose to limit the number of entries to 128, and the maximum name length to 64. Since there should realistically be few tasks that need to act as servers and thus have names, this is sufficient.

K1

TaskDescriptor

TaskDescriptor is the handle to an allocated task. It contains all the meta data such as the taskId, priority, parent, runState, stackPointer, etc. Added in K2: messageControlBlock and sendWaitQueue. The latter holds all the tasks that are waiting to be sent to this task.

TaskScheduler

TaskScheduler is a singleton class that provides operations to manipulate the task queues.

StackContext

A placeholder class to define the saved context layout of each user task.

RoundRobinQueue

A circular linked list that iterates through all the items in a round robin format. Internally, it stores a set of TaskDescriptor, and uses TaskDescriptor's intrusive data structure's link field to form an order.`

MultiLevelQueue<T>

A template class that holds an array of task queues indexed by task's priority. It is a wrapper class that has the same operations as the internal queue, and selects the corresponding queue based on task's priority.

SlabAllocator<T, SIZE>

A template class that allocates an array of fixed size memory blocks. Internally it uses a free list to track the available memory blocks. Each freed memory block is an intrusive data structure, and are linked together via an internal linkage. When the memory block is allocated, it is reinterpreted as an array of unused bytes.

Control Flow

K4

The K4 kernel does not require significant changes in control flow from K3, as we only implement new event/interrupt types. The burden of using these new interrupt types is on the I/O and CAN servers, which were described in the K4 subsection of the “Components” section above.

The three new event types are “UartRX”, “UartTX”, and “CanIO”. “UartRX” and “UartTX” are tied to the UART interrupts. RX and TX are enabled/disabled separately by setting/clearing the appropriate bits of the UART IMSC register. When a UART interrupt arrives, the kernel determines if it was an RX or TX event, unblocks the corresponding tasks, and disables the corresponding interrupt type.

CAN Interface

There are a couple of reasons why there is only 1 event type, “CanIO”, for CAN bus I/O. As described before, there is only 1 notifier for the CAN server, whose only job is to notify the CAN server when anything changes state. It is up to the server, not the kernel or the notifier, to check the status, see what changed, and what needs to be done. This is to ensure that SPI I/O only happens in one place, to prevent interleaving of transactions. Another consequence of this is that the kernel cannot selectively enable/disable RX and TX interrupts, as this involves SPI transactions. Instead, we choose to toggle interrupts at the GPIO level, which toggles all CAN I/O interrupts. Therefore, there is only 1 event type.

- `int ReceiveCAN(int tid, marklin::MMessage& msg)`
 - Wrapper for Send-ing a ReceiveRequest to the CAN server
- `int TransmitCAN(int tid, const marklin::MMessage& msg)`
 - Wrapper for Send-ing a TransmitRequest to the CAN server

Since our kernel and user tasks are both C++, we opt to use references to the `marklin::MMessage` struct in our interface.

One other change from K3 is how idle time is displayed. Previously, it was printed by the kernel. However, to prevent interleaving of ANSI escape sequences, this was changed in K4 into a dedicated GetIdle syscall, so that user tasks can query for the idle percentage and print it at their own accord.

K3

GICC and GICD configuration

The kernel main entry initializes GICD_CTLR and GICC_CTLR to enable the interrupt controllers as a sanity check. It then routes the interrupt to CPU core 0 by masking the GICD_ITARGETSR register. Finally, it enables the GICC interrupt interface through the GICD_ISENBLER register.

AwaitEvent

When task 1 calls the AwaitEvent(id) syscall, the program switches to the kernel mode.

If the id is not recognized by the kernel as corresponding to a valid event, then it immediately returns -1.

If the id is recognized by the kernel, then the TaskManager moves the task from readyQueue to the eventBlockedQueue[id].

preIrqEntry

A new entry to the vector table is added alongside the existing syscall entry for IRQs. This performs the exact same context saving as its counterpart for syscalls, except it branches to irqEntry(StackContext*) instead of syscallEntry(StackContext*).

irqEntry(StackContext* sp)

The irqEntry performs the following operations:

1. Updating the current TaskDescriptor's stack pointer to the sp arguments similar to the system call handler.
2. irqEntry reads GICC_IAR register to get the interrupt event id and set the interrupt state to active.
3. irqEntry compares the interrupt id and calls the corresponding handler. For instance, timer interrupts wake up all the timer-event blocked tasks to the readyQueue, and resets the timer

channel to the next absolute tick. The next tick is computed as an offset from the previous tick, not the time of handling, so our timer does not lose time.

4. `irqEntry` deactivates the interrupt via the `GICC_EOIR` register, and then switches to the next task as determined by the scheduler.

Idle Task

The idle task is created by the kernel and simply executes the `"wfi"` instruction in an infinite loop. Its `tid` is saved by the kernel, and on context switch, if the previous task was the idle task, it adds the time since the last context switch to an accumulator. Once a 500ms window elapses, on the next context switch, the kernel prints the percent of time spent by the idle task.

K2

Message Queue Design

At any given time, a task can only be in one of the following queues: the kernel `readyQueue`, or other task's `sendWaitQueue`. If the task is in the `RECEIVE_BLOCKED` or `REPLY_BLOCKED` state, it will simply be removed from the `readyQueue`.

Send

When task 1 sends a message to task 2, it enters the kernel, and the kernel sets task 1's `MessageControlBlock` to `Send()`'s arguments.

If task 2 is in the `RECEIVE_BLOCKED` state, then the kernel copies task 1's message to task 2's receive buffer, removes task 1 from the `readyQueue` and sets its state to `REPLY_BLOCKED`, and re-adds task 2 to the `readyQueue` and sets its state to `READY`.

If task 2 is not in the `RECEIVE_BLOCKED` state, the kernel moves task 1 from the `readyQueue` to task 2's `sendWaitQueue`.

Receive

When task 2 calls `receive`, it enters the kernel, and the kernel sets task 2's `MessageControlBlock` to `Receive()`'s arguments.

If task 2's `sendWaitQueue` is empty, then the kernel removes task 2 from the `readyQueue` and sets its state to `RECEIVE_BLOCKED`.

If task 2's `sendWaitQueue` is not empty, say task 1 is at its head, then the kernel copies the task 1's message to task 2's receive buffer. It removes the first task (task 1) from the `sendWaitQueue` and sets its state to `REPLY_BLOCKED`.

Reply

When task 2 calls `reply()`, it enters the kernel, and the kernel sets task 2's `MessageControlBlock` to `Reply()`'s arguments.

If the `tid` refers to a task 1 that is in the `REPLY_BLOCKED` state, then the kernel copies task 2's message to task 1's receive buffer, and re-adds task 1 to the `readyQueue` and sets its state to `READY`.

If the `tid` does not reference a valid task in the `REPLY_BLOCKED` state, an error is returned.

K1

Task Creation

The kernel create a task with the following procedure:

1. Allocate a task id using TidAllocator. Internally it uses a stack data structure to track the freed task id.
2. Allocate a stack frame implicitly by indexing into an array of pre-reserved stack frames with the allocated task id. Each stack frame size is 1 MB.
3. Allocate a TaskDescriptor implicitly by indexing into an array of pre-reserved TaskDescriptors with the allocated task id.
4. Pretend the beginning of the stack frame already contains the task context, including registers x0 - x30, ESR_EL1, SPSR_EL1, ELR_EL1.
5. Initialize the link register to the Exit() system call, ELR_EL1 to the function entry, and all other registers to 0.
6. Initialize the TaskDescriptor's fields, and enqueue into the scheduler queue.

Context Switching

1. In the boot loader code, the boot loader sets the exception vector to a dedicated vector with synchronized exception pointing to a function named preSyscallEntry.
2. When a system call is called, the system call calls SVC #N, where N depends on the type of the system call.
3. SVC #N looks up the exception vector, and jumps to preSyscallEntry and switches to the EL1 mode.
4. preSyscallEntry temporarily switches to EL0 stack pointer, and pushes the current task's context into its dedicated stack. It then switches back to EL1 stack pointer, and calls the syscallEntry(taskStackPointer).
5. syscallEntry(taskStackPointer) is a C function that checks ESR_EL1 to see the svc code, and performs the corresponding system calls operations such as Create, MyTid, Yield etc. ESR_EL1

and any arguments are read from the user stack. At the end of the `syscallEntry`, it schedules the next task in a round robin format.

6. Context switches to the next scheduled task by temporarily switching to `EL0` stack pointer, and pops the context registers from the task's stack. Set `ELR_EL1` to the PC to jump to the address with `ERET`.

`::Create(priority, function)`



`SVC #0`



`svcEntry:`

save task context

branch to `syscallEntry`



`syscallEntry:`

depends on the svc number, it forwards the call to the function that performs the actual operation, e.g. `syscall_handler::Create(priority, function)`



call `TaskScheduler` to schedule the next task.



context switch into the next task

Kernel State

We adopted the “temporary stack” kernel design, where the kernel acts as an event handler. As such, the function call that context switches to a task never returns. And, in `svcEntry`, the assembly resets the kernel stack pointer back to its initial value. All kernel states that persist between kernel entries are kept as static variables.